WEEK 2 · LESSON 7 OF 24 · HANDOUT

Building Models with nn.Module and the Training Loop

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=rrekAv9Fml4 — open the lesson online to watch inside the course.

🎓 **Go deeper:** freeCodeCamp: PyTorch for Deep Learning & Machine Learning — full course — https://www.youtube.com/watch?v=V_xro1bcAuA

🎓 **Go deeper:** EuroPython: How to train an image classifier using PyTorch — https://www.youtube.com/watch?v=eDRck258bnw

Your goal in this lesson: Define networks with `nn.Sequential` and `nn.Module`, write the standard PyTorch training loop with an optimiser, and track training and validation results every epoch.

Stop managing weights by hand

In Week 1 you created every weight tensor yourself. PyTorch's `torch.nn` package does that for you: a layer such as `nn.Linear(64, 32)` creates and initialises its own weights and bias, and an **optimiser** applies the gradient-descent update to all of them.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X, y = digits.data / 16.0, digits.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0,
stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2,
                                                  random_state=0, stratify=y_train)
to_x = lambda a: torch.tensor(a, dtype=torch.float32)
to_y = lambda a: torch.tensor(a, dtype=torch.long)
Xtr, Xva, Xte = to_x(X_train), to_x(X_val), to_x(X_test)
ytr, yva, yte = to_y(y_train), to_y(y_val), to_y(y_test)
print(len(Xtr), "train,", len(Xva), "validation,", len(Xte), "test")
```

We now keep **three** sets: training (to learn), **validation** (to watch progress and make choices) and test (for the final, honest score) — as in Machine Learning Fundamentals.

nn.Sequential: layers in a row

```
torch.manual_seed(0)
model = nn.Sequential(
    nn.Linear(64, 64),
    nn.ReLU(),
    nn.Linear(64, 10),        # 10 logits, one per digit
)
print(model)
print("parameters:", sum(p.numel() for p in model.parameters()))
```

64 × 64 + 64 + 64 × 10 + 10 = **4,810** learnable numbers, created and initialised automatically.

nn.Module: your own model class

For anything more than a simple stack, write a class. `__init__` creates the layers; `forward` says how data flows through them:

```
class DigitNet(nn.Module):
    def __init__(self, hidden=64):
        super().__init__()
        self.hidden = nn.Linear(64, hidden)
        self.out = nn.Linear(hidden, 10)

    def forward(self, x):
        x = F.relu(self.hidden(x))
        return self.out(x)

net = DigitNet()
print(net(Xtr[:3]).shape)     # 3 examples -> 3 rows of 10 logits
```

You call the model like a function — `net(x)` — and PyTorch runs `forward`.

The training loop

Every PyTorch project uses the same five lines per batch:

```
torch.manual_seed(0)
model = DigitNet()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

def accuracy(m, X, y):
    m.eval()
    with torch.no_grad():
        return (m(X).argmax(dim=1) == y).float().mean().item()

history = {"train_loss": [], "val_loss": [], "val_acc": []}
for epoch in range(30):
    model.train()
    perm = torch.randperm(len(Xtr))
    for start in range(0, len(perm), 32):
        idx = perm[start:start + 32]
        optimizer.zero_grad()                    # 1. clear old gradients
        logits = model(Xtr[idx])                 # 2. forward pass
        loss = F.cross_entropy(logits, ytr[idx]) # 3. loss
        loss.backward()                          # 4. backpropagation
        optimizer.step()                         # 5. update every weight

    model.eval()
    with torch.no_grad():
        history["train_loss"].append(F.cross_entropy(model(Xtr), ytr).item())
        history["val_loss"].append(F.cross_entropy(model(Xva), yva).item())
    history["val_acc"].append(accuracy(model, Xva, yva))

print("validation accuracy after 30 epochs:", round(history["val_acc"][-1], 3))
```

Learn these five steps by heart: **zero_grad, forward, loss, backward, step**. They are the same for a 5,000-parameter digit model and a 70-billion-parameter language model.

train() and eval() modes

`model.train()` and `model.eval()` switch some layers between behaviours — **dropout** (Lesson 9) and **batch normalisation** (Lesson 10) act differently while training. Always call `model.eval()` before measuring or predicting, and wrap predictions in `torch.no_grad()` to save memory and time.

Plot the learning curves

```
plt.plot(history["train_loss"], label="training loss")
plt.plot(history["val_loss"], label="validation loss")
plt.xlabel("epoch")
plt.legend()
plt.title("Learning curves")
plt.show()
```

Both curves should fall together. If the training loss keeps falling while the validation loss turns upwards, the model is **overfitting** — Lesson 9.

Predict on new data

```
model.eval()
with torch.no_grad():
    probs = F.softmax(model(Xte[:5]), dim=1)
print(probs.argmax(dim=1).tolist(), "true:", yte[:5].tolist())
print("confidence:", [round(c, 2) for c in probs.max(dim=1).values.tolist()])
print("test accuracy:", round(accuracy(model, Xte, yte), 3))
```

Save and load

```
torch.save(model.state_dict(), "digitnet.pt")      # only the learned numbers
again = DigitNet()
again.load_state_dict(torch.load("digitnet.pt"))
print("same answers:", torch.equal(again.eval()(Xte), model.eval()(Xte)))
```

A **state_dict** holds the weights; you rebuild the same class and load them in.

Key terms

- **nn.Module** — the base class for PyTorch models; `forward` defines the computation.
- **nn.Sequential** — a quick way to stack layers.
- **Optimiser** — applies the weight updates (SGD, Adam…).
- **Training loop** — zero_grad, forward, loss, backward, step.
- **train() / eval()** — switch layer behaviour for training or prediction.
- **state_dict** — a model's learned weights, for saving and loading.

Remember this: In PyTorch you describe the model once with nn.Module, and every training step is the same five lines: zero_grad, forward, loss, backward, step.

Name: _____ Date: _____

Activity

Task 1

Change the hidden size to 16 and to 256. How do the parameter count and the validation accuracy change?

Task 2

Add a second hidden layer to `DigitNet` and train it again.

Task 3

Remove `optimizer.zero_grad()`. Describe what happens to the loss.

Task 4

Print the five test digits the model is least confident about. Are they messy?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does nn.Linear(64, 10) create?

   A. A dataset
   B. An optimiser
   C. A loss function
   D. A layer with its own 64 × 10 weights and 10 biases

2. In an nn.Module class, what does the forward method define?

   A. How to save the model
   B. How data flows through the layers
   C. The learning rate
   D. The test set

3. What is the correct order of the training step?

   A. loss, forward, step, backward, zero_grad
   B. step, backward, loss, forward, zero_grad
   C. forward, step, zero_grad, loss, backward
   D. zero_grad, forward, loss, backward, step

4. What does optimizer.step() do?

   A. Evaluates the model
   B. Updates every weight using its gradient
   C. Loads data
   D. Computes gradients

5. Why call model.eval() before measuring accuracy?

   A. Some layers such as dropout behave differently during training
   B. It is only for GPUs
   C. It makes the model learn faster
   D. It deletes gradients

6. What does model.state_dict() contain?

   A. The loss history
   B. The training data
   C. The source code
   D. The model's learned weights and biases

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.